

【提案プロジェクト詳細】

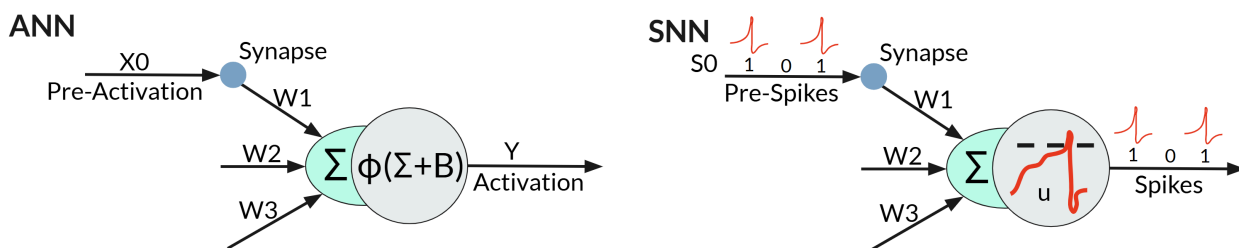
1. 何を作るか

Spiking Neural Network (SNN) とは

Spiking Neural Network (SNN) と対比して、本稿では現在広く用いられている連続値を扱う Artificial Neural NetworkをANNと呼ぶことにする。ANNとSNNの違いは、情報表現と計算の方法にある。ANNでは、各ニューロンは入力の重み付き和を計算し、それに活性化関数を適用することで連続値の出力を生成する。すなわち、入力信号は実数値として扱われ、ネットワーク内の各層で連続的な値が伝播していく。このような計算モデルは現在の深層学習の多くで用いられており、画像認識や自然言語処理などの分野で高い性能を示している。

一方、SNNではニューロンはスパイクと呼ばれる離散的なイベントによって情報を伝達する。ニューロンは入力スパイクによって膜電位が時間的に積分され、一定の閾値を超えたときにスパイクを発生させる。このスパイクが次のニューロンへ伝達されることで情報が伝搬する。そのため、SNNでは信号は連続値ではなく時間とともに発生するスパイク列として表現される。

このように、ANNが連続値の活性化を用いた静的な計算モデルであるのに対し、SNNは時間発展する膜電位とスパイクイベントに基づく動的な計算モデルである点に大きな特徴がある。下図はANNとSNNのイメージ図である。



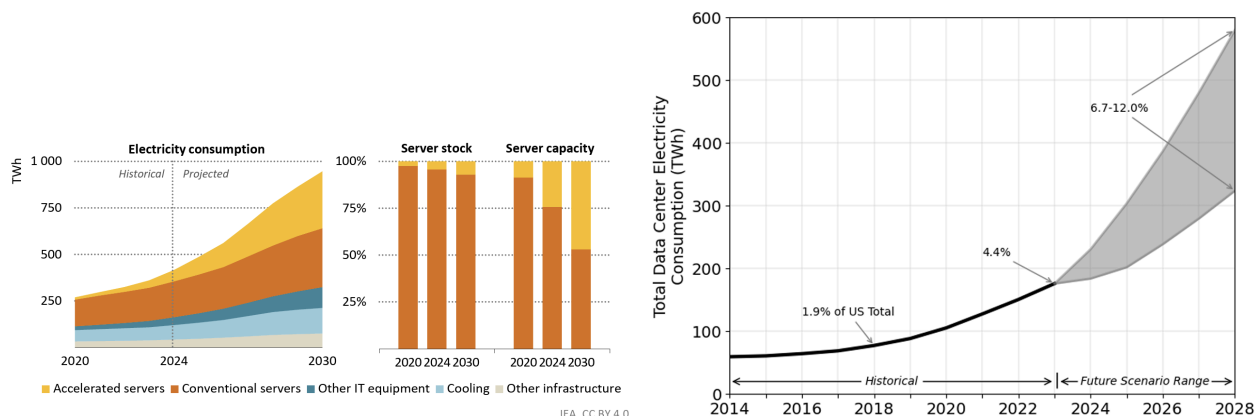
SNNは大きく分けて2種類存在する。ANNと同様に誤差逆伝播を使う勾配ベースSNNと、より実際の脳に近い学習則を使う局所学習SNNである。勾配ベースSNNは現状でもANNと同等の精度を出すことができるというメリットはあるが、ネットワークの構造や性質もANNと似ているため、以下に述べるSNN特有の特長がかなり限定的であるという問題がある。局所学習SNNは学習アルゴリズムが誤差逆伝播法ほど確立してはいないため、精度の面で劣るが、以下に述べるようにSNN特有の特長を持つ。そのため、本提案書では、局所学習SNNを指してSNNと呼ぶ。

SNNの長所

ANNと比較して、SNNには次のような長所がある。

- スパイクによるイベント駆動で計算でき、低消費電力での処理が可能[1]
 - 入力が0/1なので積和が積を省いた和に寄りやすい
 - スパイクのタイミングでのみ計算が起こるため、活動・計算・メモリアクセス・更新が疎になる
 - 時間を追加次元として使い、フレーム駆動より疎に情報を保持でき、低電力につながる
 - GPU上のTensorFlow実装と比較して、ニューロモルフィックコンピュータ(Intel Loihi)上で実行されたSNNは、推論1回あたりのエネルギー(J/Inference)基準で約110倍の電力(エネルギー)効率となったことが報告されている[2]
- 既存のANNよりも、必要なニューロン数の観点でより表現力が高い場合がある[3]
 - スパイクのタイミングにも情報を載せることができる
- オンライン学習や継続学習を既存のANNよりも自然に実装できる[4][5]
 - pike-Timing-Dependent Plasticity (STDP)により、ネットワークを局所的に、継続して学習可能

現在、ANNは性能向上と引き換えに電力需要を急増させ、アメリカなど局所的には電力・資源・インフラ制約を社会問題として顕在化させている。例えば、下図左はInternational Energy Agencyによる試算[6]だが、データセンターの電力消費は2030年に向けて急増し、特に増えるのはGPU等のAI向けアクセラレータサーバであるとされている。さらに、下図右はLawrence Berkeley National Laboratoryの試算[7]で、米国データセンターの総電力消費推移を表しているが、2028年にかけて米国全体の電力消費に占めるデータセンターの割合が最大12%にまで増加する可能性があると考えられている。SNNは低消費電力であるため、電力需要の急増という社会問題に対する解決策として考えられているだけでなく、表現力の高さやより人間の脳に近い学習則の実装により、将来的に既存のANNより良い選択肢になりうると考えられている[8][9]



SNN研究が抱えている課題

局所学習SNNについてはまだまだ研究途中である。ここで、現在のSNN研究には検証用のニューロモルフィックハードウェアと、シミュレーション用のGPUやCPU等が用いられる。Loihi (Intel) や NorthPole (IBM) など、SNNを完全にイベント駆動で実行できるニューロモルフィックハードウェアはSNNの省電力性をフルに発揮でき、実時間での完全イベント駆動の検証用に価値があるが、例えばIntelのLoihiは市販していなかったりと、入手が難しく、実時間での検証のため高速ではない。さらに、書けるモデルや学習則の自由度が一般のPython実装よりも狭く、用意された学習エンジンの枠・ニューロン数/コア数などの制限がある。そのため、大半の研究者はGPUやCPU等を使ってタイムステップを切ってシミュレーションを行って研究をしている。しかし、GPUやCPUには以下のような問題がある。

GPUはPyTorchなどのエコシステムを利用でき、大規模並列計算により大規模ネットワークのシミュレーションを高速に実行できるという利点がある。一方で、GPUは本来、密な行列計算のように規則的で同一の演算を大量のデータに対して適用する処理を高速化するように設計されたアーキテクチャである。これに対し、SNNの計算はスパイク発生に応じて処理対象が変化し、接続構造に従って不規則なメモリアクセスが発生するため、計算の並列性やメモリアクセスの規則性が低い。そのためGPUのSIMD型並列処理やメモリアクセス最適化と根本的に相性が良くなく、演算ユニットやメモリ帯域を十分に活用できない場合が多い。このように、GPUはSNNの計算構造そのものに適したアーキテクチャとは言い難い。

CPUは柔軟な制御構造を持ち、任意のニューロンモデルや学習則を自由に実装できるため、アルゴリズム検証の基盤として広く利用されている。また、不規則なメモリアクセスにも比較的強く、複雑なイベント処理や局所学習則をそのまま実装できるという利点がある。しかし、局所学習を伴う疎SNNでは、1回のスパイク更新あたりの演算量は小さい一方で、接続情報・重み・ニューロン状態・トレースなど多数のデータの読み書きが発生する。そのため性能は演算能力よりもメモリアクセスに強く依存する傾向があり、ネットワーク規模が大きくなると計算時間が急激に増大する。

このようなGPU/CPUの問題に対して、専用ハードウェアによる高速化も研究されており、例えば大規模SNNシミュレーション向けの専用計算機であるPOETSでは、一般的なCPUベースのシミュレーションフレームワークであるBrianと比較して最大20倍程度の高速化が報告されている[10]。このことは、SNNの計算を汎用CPUの実行モデルにそのまま載せるのではなく、計算構造に合わせてハードウェアを設計することで大きな性能向上が得られる可能性を示している。しかし、これらの研究の多くは特定のハードウェアや研究目的に合わせて個別に設計されたシステムであり、一般の研究者が広く利用できる共通の計算基盤としては十分に整備されておらず、研究成果やソフトウェア資産を広く共有することが難しい。

このように、現在のSNN研究ではCPUやGPUを用いたシミュレーションや、特定の用途に特化した専用ハードウェアなど、さまざまな手法が個別に研究されている。しかし、これらはそれぞれ独立した形で発展しており、SNNの計算構造に最適化された共通の計算基盤として広く利用されているものは存在していない。

深層ニューラルネットワークの分野では、GPUという計算機アーキテクチャと、それを効率的に利用するためのCUDAコンパイル基盤、さらにPyTorchなどの高水準フレームワークが一体となったエコシステムが整備されたことで、大規模な研究開発が進展するとともに、NVIDIAがその地位を確固たるものにしている。一方でSNNの分野では、スパイク伝播や局所学習といった計算構造に適したハードウェアと、それを効率的に利用するコンパイラ、さらに研究者が容易に利用できるPythonフレームワークを含めた統合的な計算基盤が存在していない。

すなわち、SNN研究における大きな課題の一つは、ANNにおけるGPU-CUDA-PyTorchのように、ハードウェア・コンパイラ・高水準フレームワークが一体となった、SNNに最適化された共通の計算プラットフォームが存在していないことである。

本プロジェクトで作るもの

以上で述べた現在のSNN研究の問題を解決するため、私は本プロジェクトで「局所学習SNNを高速実行できる専用計算基盤」を作ることを目指す。具体的には、Pythonフレームワーク、SNNアクセラレータ、そしてそれらをつなぐコンパイラ・ランタイム・ツールを作ることを目指す。本プロジェクトは最終的には今のANN研究でGPUを使うように簡単に、汎用的に使えるものを作ることが目標であるが、未踏期間中にそこまで完成させることは現実的ではないと考えている。そのため、未踏期間中には汎用性を落としながらも研究者が実際にPythonから使えるようなβ版を作成することを目指し、以下の三つの要素から構成されるシステムを開発する。

(1) Pythonフレームワーク

本プロジェクトでは、研究者が局所学習SNNのモデルをPythonで記述できるフレームワークを開発する。既存のSNNシミュレーション環境であるBrianが備える柔軟なモデル記述や観測機構の考え方を参考にしながら、局所学習SNNの研究に必要な機能に焦点を当てた軽量なフレームワークとして設計する。

この1年間では、ニューロンモデル、シナプス結合、発火条件、リセット則、遅延、局所学習則をPythonコードとして記述できる基本的なAPIを設計する。具体的には、ニューロン集団、シナプス接続、外部入力、発火条件、膜電位更新、重み更新といった要素をPythonオブジェクトとして定義し、それらを組み合わせることで局所学習SNNのネットワーク構造を記述できるようにする。これにより、研究者は回路構成やメモリ配置といったハードウェア詳細を意識せずに、SNNモデルをPythonコードとして定義できる。

さらに、スパイク列、膜電位、重み、シナプストレース、イベント処理数といったネットワーク内部状態を取得する観測機能を実装し、学習過程やネットワークダイナミクスをPython側から解析できるようにする。また、実験条件の設定や実行制御をPythonから行えるようにし、ネットワーク構造、入力データ、実行時間といった実験設定をコードとして管理できる環境を整備する。これにより、局所学習SNNの挙動を観測しながら実験を行うための研究用フレームワークを構築する。

(2) コンパイラ・ランタイム・ツール

Pythonで記述されたSNNモデルをFPGA上で実行するため、本プロジェクトでは専用のコンパイラおよび実行ツールを開発する。完全な汎用コンパイラの実装ではなく、局所学習SNNをFPGA上で実行するために必要な処理に焦点を当てたプロトタイプとして設計する。

この1年間では、Pythonフレームワークで記述されたネットワーク構造を解析し、ニューロン集合、シナプス接続、イベント処理、学習処理、状態変数、メモリ配置情報から構成される中間表現を生成するコンパイラを実装する。この中間表現は、SNNの計算構造をハードウェア非依存の形で表現するデータ構造であり、ニューロン更新、シナプス伝播、局所学習更新といった計算単位、およびそれらの実行順序やデータ依存関係を明示的に保持する。

その後、この中間表現をFPGA上のメモリ構造および実行モデルに合わせて変換し、接続情報、重み、ニューロン状態をハードウェアにロードするためのバイナリデータとメタデータを生成する処理を実装する。

さらに、FPGAへのモデルロード、実行開始、データ転送、結果回収を行うランタイムを開発し、イベント処理数、キュー長、メモリアクセス回数といった実行統計を取得できる計測機能を整備する。これにより、SNNの実行過程を詳細に観測し、ハードウェア上での計算挙動を分析できる実験環境を構築する。

(3) SNNアクセラレータ(FPGA)

局所学習SNNをFPGA上で効率よく実行する専用アクセラレータを実装する。本アクセラレータは、スパイクイ

ベントに応じた接続探索、ニューロン状態更新、局所学習更新を効率よく処理できる構造を持つ。

アクセラレータは、スパイクイベントを管理するイベントキュー、シナプス接続を参照してスパイク伝播を計算する接続探索回路、膜電位や発火状態を更新するニューロン状態更新回路、局所学習則に基づいて重みを更新する学習更新回路から構成される。イベントキューには発火イベントが蓄積され、各イベントに対して接続情報を参照しながらシナプス伝播を行い、対象ニューロンの状態更新および学習更新が順次実行される。

接続情報、シナプス重み、ニューロン状態、シナプスストレス、遅延バッファといったネットワーク状態は、FPGA上のオンチップメモリに保持する構成とする。これにより、スパイクイベントを起点とした不規則な接続探索や状態更新を低レイテンシで処理できるようにする。本プロジェクトでは扱えるネットワーク規模に制限を設けることで、すべての状態をオンチップメモリに配置し、外部メモリアクセスによる性能低下を避ける構成を採用する。この構成により、局所学習SNNに特徴的な疎接続計算と小粒度更新をFPGA上で効率よく実行できるアクセラレータを構築する。

さらに、本基盤を一般の研究者が利用可能とするため、開発した実行環境をサーバ上に配置し、遠隔からアクセラレータを利用できる実行環境を整備する。研究者はPythonフレームワークからモデルを記述し、サーバにジョブを送信することで、FPGA上のアクセラレータを用いた計算を遠隔から実行できるようにする。これにより、専用ハードウェアを所有していない研究者でも本基盤を利用した実験を行うことが可能となり、局所学習SNN研究の計算基盤としての普及を促進する。

作成したプロトタイプ

2. どんな出し方を考えているか

本プロジェクトでは、プログラム期間中はFPGAを用いてイベント駆動型SNN計算基盤の実装と検証を行うが、これは最終目標そのものではなく、将来的な専用チップ化に向けた基盤技術の確立を目的とするものである。プログラム終了後も開発を継続し、最終的にはGPUのように研究者や開発者が広く利用できるSNN計算チップとして社会に展開することを目指す。将来的には、SNN分野において現在のNVIDIAがGPU分野で担っているような中核的な計算基盤の地位を確立することを構想している。

そのため、本プロジェクトでは普及のための層と競争力の源泉となる技術層を分けて開発を進める。研究者が直接利用するPythonフレームワークについてはオープンソースとして公開し、SNNモデルの記述、実行、観測を容易に行える研究基盤として提供する。一方で、ハードウェアアーキテクチャやコンパイラなどの低レイヤの実装については、将来的な専用チップ化と産業展開を見据えた中核技術として非公開の形で継続的に開発する。

また、本基盤の有効性については、既存の計算環境との性能比較や実験結果を論文として発表し、学術的な検証を行う。これらの活動を通じて研究者コミュニティにおける利用を促進し、SNN研究の共通計算基盤としての普及を目指す。

3. 斬新さの主張、期待される効果など

既存の SNN 専用計算基盤の先行研究として、BrainScaleS や NeuroFlow が挙げられる。

BrainScaleS-2は高速な加速型ニューロモルフィック基盤と高水準ソフトウェア環境を備えているが、アナログ神経回路による物理モデリングを採用しているため、ニューロンモデルやシナプスモデルは回路構造に依存し、任意の更新式や学習則を自由に実装することはできない。また、接続構造もハードウェアの配線ネットワークに依存する。

NeuroFlow は FPGA を用いた高速な SNN シミュレーション基盤であり、PyNN 互換の高水準インターフェースからネットワークを実行できる。しかし、ニューロンモデルや更新式はハードウェアとして合成されるため、モデルを変更するたびに FPGA の再合成が必要となり、アルゴリズムの試行錯誤を高速に行う研究用途では大きな制約となる。

本プロジェクトの成果により、研究者は局所学習SNNのモデルをPythonで記述し、そのまま専用アクセラレータ上で高速に実行できる計算基盤を利用できるようになる。これにより、疎接続やイベント駆動といったSNN特有の計算構造に適した形で大規模ネットワークのシミュレーションや学習実験を高速に行うことが可能となり、局所学習SNNのアルゴリズム研究を大きく加速することが期待される。

4. 具体的な進め方と予算

プロトタイプまで書けたらかな

5. 提案者の腕前を証明できるもの

MITでのGPUの開発

あとは今までの経験

6. プロジェクト遂行にあたっての特記事項

研究室は未定であるが、卒業研究のための研究室に配属予定である。

8月ごろに大学院の入学試験を受験予定であるが、卒業研究のために配属される研究室にそのまま進学する予定であり、大きな負担ではないと考えている。

7. IT以外の勉強、特技、生活、趣味など

大学では神経科学とコンピュータサイエンスを学んでいる。特に、人間の知能や思考の仕組みを計算モデルとして理解し、それをコンピュータ上で再現することに興味がある。卒業研究のための研究室配属の直前というタイミングであり、脳の生理学的な詳細そのものよりも、神経回路がどのような計算原理によって学習や推論を実現しているのかという点に関心があり、SNNなどの脳に着想を得た計算モデルを通して、汎用的な知能の実現につながるアルゴリズムや計算基盤について研究したいと考えている。

「勉強は答えがあるから簡単」というスタンスで生きてきたため、いわゆる勉強は得意であり、筑波大学附属駒場高校を卒業生総代として卒業している他、MITへの留学でもComputational cognitive scienceの授業でA+をとるなどGPAは4.7/5.0であった。

また、小学生の時から高校までサッカーをしており、大学入学後は東京大学ア式蹴球部でアナリストとして自チームや相手チームの分析を行っている。中学生からはジャグリングを趣味としており、7つのボールを一度に扱える。

8. 将来のITについて思うこと・期すること

SNNが時代を作るんじゃないっすかね？

9. 参考文献

[2]<https://arxiv.org/abs/1812.01739>

```
[10] @inproceedings{rhodes2020spiking,
  title={Spiking Neural Networks on the POETS Architecture},
  author={Rhodes, Benjamin and Orchard, Garrick and Furber, Steve and Brown, Andy},
  booktitle={International Conference on Neural Information Processing},
  pages={463--475},
  year={2020},
  publisher={Springer}
}
```

左上にプロジェクト名及び申請者名を記入後、下記の事項に関して、合計10ページ以内にまとめて記入して下さい

さい。

- 1 なにをつくるか
- 2 どんな出し方を考えているか
- 3 斬新さの主張、期待される効果など
- 4 具体的な進め方と予算
- 5 提案者(たち)の腕前を証明できるもの
- 6 プロジェクト遂行にあたっての特記事項
- 7 IT以外の勉強、特技、生活、趣味など
- 8 将来のITについて思うこと・期すること

A4サイズ、縦置き・横書きで10.5ポイント以上の文字サイズで、定型フォームのワープロソフト、あるいはPDF、RTFなどの汎用的なファイル形式で記入して下さい。

詳細は、公募要領を参照してください。